

ABSTRACT

Water is a finite resource, yet overhead storage tanks in domestic, institutional and small industrial settings are still managed almost entirely by manual observation. Users climb to inspect tank levels, guess when to switch on a pump, and frequently discover the consequences of a wrong guess only after water has overflowed onto a roof or a motor has run dry against an empty sump. Both outcomes carry a real cost: wasted water, wasted electricity, avoidable structural damage, and premature failure of the pump itself.

This project presents an IoT-based Smart Water Level Monitoring and Automated Pump Control System built around an ESP32 microcontroller and the Blynk IoT platform. A non-contact HC-SR04 ultrasonic sensor is mounted at the top of the tank and measures the air gap between the sensor face and the water surface. The firmware converts this air gap into a water depth, classifies the depth into four discrete states, and reports the result simultaneously through three independent channels: a 16x2 I2C liquid crystal display for on-site reading, a four-segment LED bar for immediate at-a-glance status, and the Blynk cloud dashboard for remote monitoring from any location with internet access. A five-volt relay module, galvanically isolated from the control electronics, allows the user to switch a DC water pump on or off from the same dashboard.

The prototype was assembled on a breadboard, powered by two Li-ion cells in a battery holder, and tested by simulating rising and falling water levels. Results show that the system reliably tracks the water level, automates pump control to prevent overflow and dry-running, and reduces the manual effort required for day-to-day tank monitoring. This report documents the objectives, requirements, hardware and software design, circuit implementation, Blynk configuration, testing results, and scope for future enhancement of the system.

Keywords: Internet of Things, Smart Water Management, ESP32, Blynk IoT, Ultrasonic Sensor, Automated Water Pump, Embedded Systems.

TABLE OF CONTENTS

Abstract	ii
Chapter 1: Introduction	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives of the Project	4
1.4 Scope of the Project	5
Chapter 2: Literature Review	6
2.1 Level Sensing Techniques	6
2.2 Controller Platforms	6
2.3 Cloud Platforms and Dashboards	6
2.4 Identified Gaps	7
Chapter 3: System Requirements	8
3.1 Hardware Requirements	8
3.2 Software Requirements	9
Chapter 4: System Design and Methodology	10
4.1 System Architecture	10
4.2 Circuit Design and Pin Mapping	11
4.3 Circuit Diagram	12
4.4 Working Principle	13
Chapter 5: Implementation	15
5.1 Hardware Assembly	15
5.2 Blynk Application Setup	15
5.3 Firmware / Source Code (Key Snippets)	17
Chapter 6: Simulation-Based Design Validation	19
6.1 Rationale for a Simulation Stage	19
6.2 Simulation Environment and Model Construction	19
6.3 Simulation Test Procedure	20
6.4 Defects Identified and Corrected in Simulation	21
6.5 Limits of Simulation Fidelity	21
Chapter 7: Results and Discussion	23
7.1 Test Methodology	23
7.2 Measurement Accuracy	23
7.3 Classification Consistency	24
7.4 Automatic Shut-Off Interlock Verification	24

7.5 Responsiveness and Stability	25
7.6 Discussion	26
Chapter 8: Limitations	27
8.1 Sensing Limitations	27
8.2 System and Deployment Limitations.....	27
Chapter 9: Conclusion and Future Scope.....	29
9.1 Conclusion	29
9.2 Future Scope and Scalability	30
References.....	33
Appendix A:	
Complete Firmware Source Code.....	35

CHAPTER 1: INTRODUCTION

1.1 Background

Water is a resource that is frequently wasted through overflow, and equally often mismanaged through undetected shortages that leave pumps running dry. In homes, hostels, offices, and small industrial units, overhead tanks and sumps are still checked manually — either by physically inspecting the tank or by listening for the sound of an overflowing pipe — a method that is unreliable, unsafe for tall or inaccessible tanks, and wasteful of both water and electricity.

The growth of Internet of Things (IoT) technology has made it practical to replace this manual inspection with continuous, automated sensing, pump control and remote visibility. An IoT-enabled water level monitoring and automated pump control system that senses the tank level and reports it to a connected application removes the guesswork from tank management and allows the pump to be controlled automatically or from anywhere with an internet connection.

1.2 Problem Statement

Manual monitoring of water storage tanks is inefficient, imprecise and reactive. The user has no continuous knowledge of the stored volume, no automated protection against overflow or dry running, and no means of checking status without being physically present at the tank. Commercially available level monitoring instruments do exist, but they are engineered for industrial process control, are priced accordingly, and require professional installation, which places them outside the reach of individual households, small institutions and student-scale agricultural setups.

Furthermore, many low-cost hobby solutions rely on float switches or resistive probes that make direct contact with the water. Contact-based sensing introduces electrolytic corrosion of the electrodes, biofouling in stored water, and a genuine risk of contaminating potable supply. A monitoring approach that requires no contact with the water column is therefore preferable on both maintenance and hygiene grounds.

There is consequently a need for an affordable, non-contact, easily deployable water level monitoring system that provides continuous local indication, remote visibility through a standard mobile interface, and safe remote actuation of the pump.

1.3 Objectives of the Project

- To design and implement a low-cost, non-contact water level measurement node using an HC-SR04 ultrasonic sensor interfaced to an ESP32 microcontroller.
- To convert raw time-of-flight measurements into a calibrated water depth and classify that depth into four discrete, human-readable states.

- To provide redundant local indication through a 16x2 I2C LCD and a four-segment LED bar, so that the system remains informative even during a network outage.
- To transmit live level data to the Blynk IoT cloud and render it on a mobile and web dashboard accessible from any internet-connected location.
- To enable safe remote control of a DC water pump through a galvanically isolated relay module commanded from the Blynk dashboard.
- To evaluate the accuracy, classification consistency, responsiveness and stability of the completed prototype under controlled test conditions.

1.4 Scope of the Project

The scope of this project covers the design, assembly, and testing of a single-tank prototype suitable for a household overhead tank, sump, or small reservoir. It includes ultrasonic sensor integration, microcontroller programming, cloud connectivity through Blynk, LED-based local level indication, and a basic automated pump-control response. The project does not cover large-scale multi-tank municipal water distribution monitoring, though the same architecture can be extended to multiple nodes for that purpose.

CHAPTER 2: LITERATURE REVIEW

Automated liquid level measurement is a mature field in industrial instrumentation, but its migration into low-cost consumer and small-scale agricultural contexts is comparatively recent and has been driven almost entirely by the falling price of wireless microcontrollers. The literature relevant to this project divides into three strands: sensing technique, controller platform, and cloud integration.

2.1 Level Sensing Techniques

Reported approaches to tank level sensing fall broadly into contact and non-contact categories. Contact methods include float switches, conductive probe arrays and capacitive strips. These are mechanically simple and inexpensive, and float switches in particular remain common in commercial products. Their recurring weakness in the literature is degradation over time: probes corrode through electrolysis, floats jam with sediment or biofilm, and each failure mode is silent until the system misreports.

Non-contact methods include ultrasonic time-of-flight, infrared reflection, radar and pressure transduction at the tank base. Ultrasonic sensing has become the dominant choice in low-cost work because modules such as the HC-SR04 provide usable centimetre-scale resolution at a very low unit price and require only two digital pins. Reported limitations are consistent across studies: a dead zone of roughly two centimeters directly in front of the transducer, sensitivity of the speed of sound to air temperature, and spurious readings from foam, floating debris or a strongly agitated surface. Several studies address the temperature dependence by adding a companion temperature sensor and correcting the speed-of-sound constant at runtime, an enhancement identified in Chapter 9 of this report as a worthwhile extension.

2.2 Controller Platforms

Early IoT monitoring work relied on an Arduino board paired with a separate Wi-Fi or GSM shield, an arrangement that increased both cost and wiring complexity. The ESP8266 consolidated processor and radio into a single low-cost module and became the default platform for a generation of published projects, including the smart plant monitoring architecture that informed the structure of this report. The ESP32 is its successor and offers material advantages for the present application: a dual-core processor that tolerates the concurrent demands of network servicing and sensor timing, a larger complement of GPIO pins, hardware timers, and a more capable radio. Its principal constraint, noted consistently in the literature and encountered directly during this project, is that its GPIO pins are specified for 3.3 volt logic and are not tolerant of the 5 volt signals produced by many popular peripheral modules.

2.3 Cloud Platforms and Dashboards

Published implementations typically rely on one of three integration patterns: a self-hosted web server on the microcontroller itself, publication to an MQTT broker such as Mosquitto with a

separate visualization layer, or a managed platform such as Blynk, ThingSpeak or Adafruit IO. Self-hosting is the cheapest but restricts access to the local network. MQTT is the most flexible and scalable but requires infrastructure that must itself be maintained. Managed platforms trade some flexibility for a working mobile application, device provisioning and authentication out of the box, which is why Blynk is widely selected for prototype and educational work. Studies incorporating Blynk consistently report that the ability to define thresholds and receive push notifications materially improves the practical usefulness of a monitoring system relative to passive data logging.

2.4 Identified Gaps

Three limitations recur across the reviewed body of work and directly shaped the design decisions in this project.

- Reporting of electrical interfacing detail is frequently incomplete. Many published schematics connect a 5 volt sensor output directly to a 3.3 volt microcontroller pin without comment, a practice that is functionally tolerated by the ESP32 clamping structure but is outside its absolute maximum rating and constitutes a latent reliability defect.
- Blocking firmware architectures are widespread. A measurement routine placed directly in the main loop with an unbounded blocking wait will starve the network stack, producing intermittent cloud disconnection that is then misattributed to network quality rather than to firmware structure.
- Dependence on continuous internet connectivity is rarely mitigated. Systems that lose all indication when the network is unavailable are poorly suited to regions with intermittent coverage.

The present work addresses the first gap through explicit level shifting on the sensor return path, the second through a timer-driven measurement architecture with a bounded acquisition timeout, and the third through redundant local indication that remains fully functional without any network connection.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Table 3.1 – Hardware Requirements

S.No.	Component	Specification	Purpose / Function
1	ESP32 DevKit V1	30-pin, ESP32-WROOM-32, 240 MHz dual core	Wi-Fi enabled microcontroller; reads the sensor, runs the classification logic, drives all indicators and communicates with the Blynk cloud
2	HC-SR04 Ultrasonic Sensor	2 cm to 400 cm range, 3 mm resolution, 5 V supply	Non-contact measurement of the air gap between the sensor face and the water surface by acoustic time of flight
3	16x2 LCD Display	HD44780 controller, 5 V	On-site display of the classified level and current pump status
4	I2C Interface Module	PCF8574 backpack, address 0x27 or 0x3F	Reduces the LCD interface from sixteen parallel connections to two I2C lines
5	5 V Relay Module	SRD-05VDC-SL-C, 10 A contacts, opto-isolated	Galvanically isolated switch allowing the 3.3 V logic to control the pump circuit safely
6	DC Water Pump	Submersible mini pump with delivery tube	Actuator that transfers water when commanded
7	LED Indicators	5 mm, four units	Four-segment bar giving immediate visual indication of the level state
8	Resistors	180 ohm, four units	Current limiting for the LED indicators
9	Resistors	1 k Ω and 2 k Ω	Voltage divider conditioning the 5 V Echo output to a 3.3 V logic level
10	Li-ion Cells and Holder	18650, 3.7 V nominal, two units	Independent power source for the pump circuit
11	USB Power Bank	5 V regulated output	Portable supply for the ESP32 and its low-current peripherals
12	Breadboards	Full size, two units	Prototyping substrate; two boards are joined so the 30-pin ESP32 can straddle the central channel
13	Jumper Wires	Male-male, male-female, female-female	Interconnection of all modules

3.2 Software Requirements

Table 3.1 – Software and Platform Requirements

S.No.	Software / Platform	Version / Source	Purpose
1	Arduino IDE	2.x	Writing, compiling and uploading firmware to the ESP32
2	ESP32 Arduino Core	Espressif board package	Board support, toolchain and hardware abstraction for the ESP32
3	Blynk Library	Blynk by Volodymyr Shymanskyy	Client library implementing the Blynk protocol, virtual pins and timer services
4	LiquidCrystal_I2C	Frank de Brabander	Driver for the HD44780 controller over the PCF8574 I2C backpack
5	Wire Library	Arduino core	Low-level I2C bus transactions and clock rate control
6	Blynk IoT Platform	Cloud service, mobile and web app	Device provisioning, authentication, dashboard rendering and remote control
7	Wokwi Simulator	wokwi.com, browser based	Functional simulation of the complete circuit and firmware prior to hardware assembly
8	Wi-Fi Network	2.4 GHz, 802.11 b/g/n	Connectivity between the node and the Blynk cloud

CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY

4.1 System Architecture

The system's design consists of three key layers: a Sensing Layer, a Processing Layer, and an application layer that builds connection on the Blynk platform.

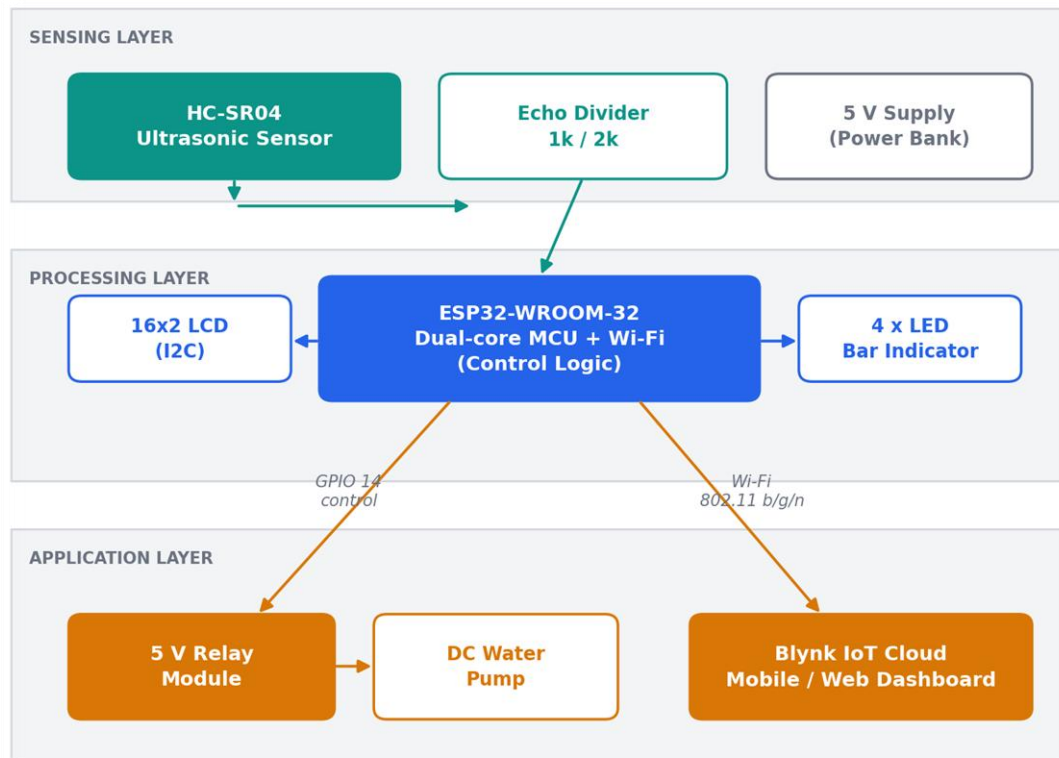


Fig. 4.1 – Three-Layer System Architecture

4.1.1 Sensing Layer

The HC-SR04 ultrasonic sensor emits a burst of eight 40 kHz pulses when its trigger input is held high for ten microseconds, then asserts its echo output for a duration proportional to the round-trip time of the reflected wave. The module is powered at 5 volts, which means its echo output also swings to 5 volts. A resistive divider of 1 kilohm in series and 2 kilohm to ground scales this to approximately 3.33 volts before it reaches the microcontroller, keeping the signal within the specified input range of the ESP32. The trigger line requires no conditioning, since a 3.3 volt output from the ESP32 exceeds the logic-high threshold of the sensor input.

4.1.2 Processing Layer

The ESP32 is the computational core. It generates the trigger pulse, times the echo return, converts the pulse width to a distance, applies the calibration constant for the specific tank, classifies the result, and drives all three output channels. Its dual-core architecture is significant here: the radio and protocol stack are serviced on one core while application code runs on the other, which reduces the interference between network activity and the microsecond-scale timing that the ultrasonic measurement depends upon.

The 16x2 LCD is driven over the I2C bus through a PCF8574 port expander, using GPIO 21 for data and GPIO 22 for clock. The four LED indicators are driven directly from GPIO 2, 4, 5 and 18, each through a 180 ohm series resistor that limits forward current to a safe value for both the LED and the pin.

4.1.3 Application Layer

The Blynk cloud receives level telemetry on virtual pin V0 and returns pump commands on virtual pin V1. The mobile and web dashboards render the level as a gauge and expose the pump as a switch widget. The relay module forms the physical boundary between the control electronics and the pump power circuit; its contacts are mechanically and electrically isolated from its coil, which is what permits the two power domains to remain entirely separate.

4.2 Circuit Design and Pin Mapping

Table 4.1 – Complete Pin Mapping

Module	Module Pin	ESP32 Pin	Direction	Notes
HC-SR04	VCC	VIN (5 V)	—	Requires 5 V; ranging degrades at 3.3 V
HC-SR04	Trig	GPIO 12	Output	No conditioning required
HC-SR04	Echo	GPIO 13	Input	Via 1k / 2k divider to 3.3 V
HC-SR04	GND	GND	—	Common ground with controller
I2C LCD	VCC	VIN (5 V)	—	HD44780 contrast requires 5 V
I2C LCD	SDA	GPIO 21	Bidirectional	Bus clocked at 50 kHz
I2C LCD	SCL	GPIO 22	Output	Bus clocked at 50 kHz
I2C LCD	GND	GND	—	Common ground with controller
Relay Module	VCC	VIN (5 V)	—	Coil supply
Relay Module	IN	GPIO 14	Output	Active low on this board
Relay Module	GND	GND	—	Coil ground only
Relay Module	COM / NO	Pump	—	Isolated pump circuit
LED 1	Anode	GPIO 19	Output	Via 180 ohm resistor
LED 2	Anode	GPIO 18	Output	Via 180 ohm resistor
LED 3	Anode	GPIO 5	Output	Via 180 ohm resistor
LED 4	Anode	GPIO 15	Output	Via 180 ohm resistor; strapping pin, see note

The LED pin assignment was revised during the simulation stage. The original allocation used GPIO 2 and GPIO 4, both of which are strapping pins sampled at reset to select the boot mode.

Although an LED with a series resistor to ground holds such a pin in its default state and does not in practice prevent booting, occupying boot-critical pins with indicator outputs is poor practice because any later change to the network, such as the addition of a pull-up, would render the board unbootable. The revised allocation moves these two indicators to GPIO 19 and GPIO 18, which carry no boot function.

Two strapping pins remain in use and are documented here explicitly so that the constraint is not lost in a future revision. GPIO 12 carries the trigger line and is sampled at reset to select the internal flash regulator voltage; if it is held high at power-up the board will not boot. Because the trigger is configured as an output and driven low during initialisation, and because no pull-up is fitted, the pin is safely low at reset. GPIO 15 carries the fourth indicator and is sampled at reset to control the emission of the boot ROM diagnostic messages on the serial port; holding it low suppresses those messages. This does not prevent booting, but it removes diagnostic output that is useful during commissioning, and reallocating this indicator to a general-purpose pin such as GPIO 23 is recommended in any subsequent revision.

4.3 Circuit Diagram

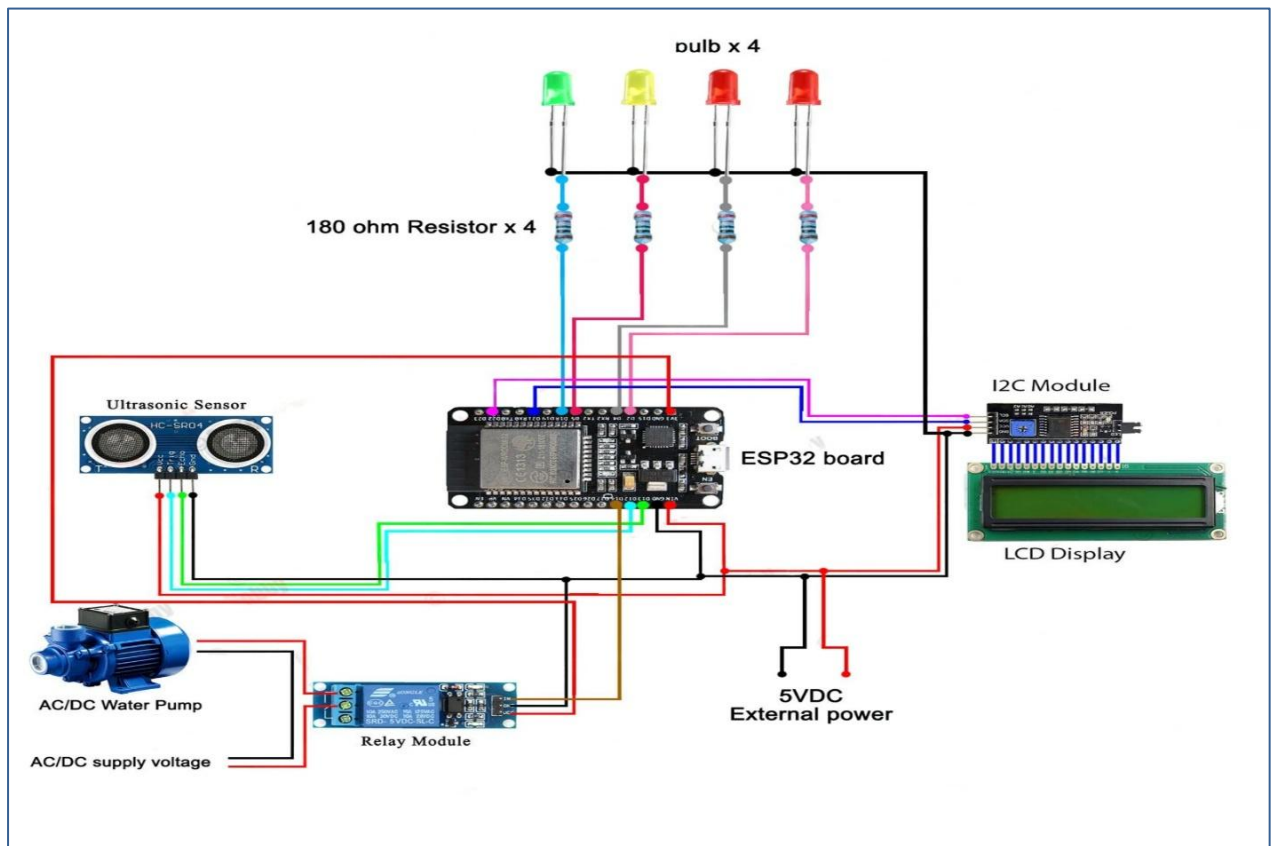


Fig. 4.2 – Circuit Diagram of the Water Level Monitoring automated pump control system

The components are wired as follows: the ultrasonic sensor's TRIG and ECHO pins connect to two digital I/O pins of the ESP32; the four LEDs, each in series with a 180 Ω resistor, connect to four separate digital output pins of the ESP32 to ground; the relay module's control pin

connects to another digital output pin of the ESP32, and its switched side connects the battery pack to the mini water pump; the 16x2 LCD connects via its I2C module to the ESP32's SDA/SCL pins; and the two breadboards are used to distribute power rails and prototype the sensor/LED/relay wiring conveniently. The complete assembly is powered by the two Li-ion cells seated in the battery holder.

4.4 Working Principle

The system executes a repeating sense, classify, publish and act cycle. Measurement is scheduled by a software timer at a one second interval rather than being invoked directly from the main loop, and the echo acquisition is bounded by a thirty millisecond timeout. The rationale for both decisions is developed in Section 5.4.

- Step 1: On reset, the firmware configures all pins, initialises the I2C bus at 50 kHz, discovers the LCD address by scanning the bus, and drives the relay to its inactive state so that the pump cannot start unintentionally during boot.
- Step 2: The node associates with the configured 2.4 GHz Wi-Fi network and authenticates to the Blynk cloud using its template identifier and authentication token.
- Step 3: Once per second the timer invokes the measurement routine, which issues a ten microsecond trigger pulse on GPIO 12.
- Step 4: The routine measures the width of the returned echo pulse on GPIO 13, subject to a thirty millisecond timeout. A timeout is treated as a failed acquisition; the routine logs the condition and returns without updating any output, so that a transient sensor fault cannot propagate a false reading to the display or the cloud.
- Step 5: A valid pulse width is converted to distance in centimetres using the speed of sound in air, halved to account for the round trip.
- Step 6: Water depth is computed as the calibrated tank depth minus the measured distance, clamped to a non-negative value, and published to Blynk on virtual pin V0.
- Step 7: The raw distance is classified against the thresholds of Table 4.2. The corresponding LED pattern is asserted and the corresponding text is written to the display buffer.
- Step 8: Both display rows are redrawn in full from the buffer, so that any previously corrupted character is overwritten on the next cycle.
- Step 9: Independently of the measurement cycle, a command arriving on virtual pin V1 from the dashboard switches the relay and updates the pump status line on the display.

4.5 Flowchart

Figure 4.3 summarizes the control flow, including the failed-acquisition path and the asynchronous pump command branch.

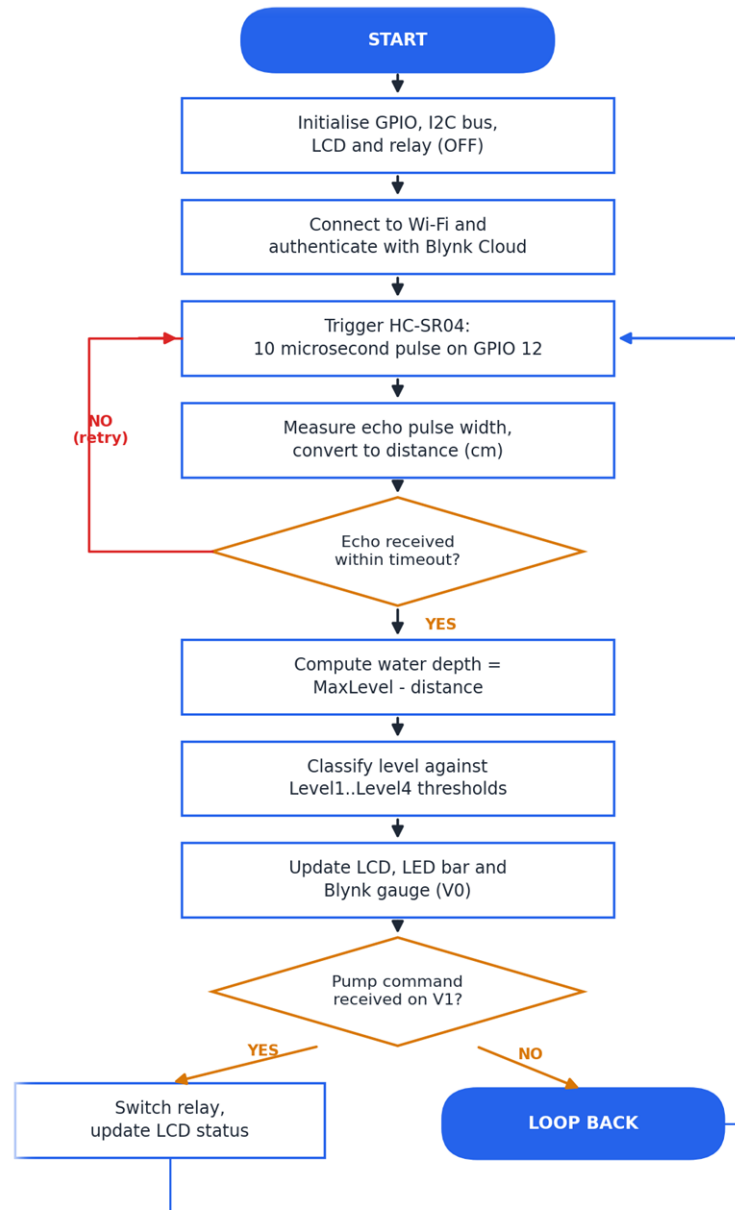


Fig. 4.3 – System Operation Flowchart

CHAPTER 5: IMPLEMENTATION

5.1 Hardware Assembly

The prototype was assembled on two full-size breadboards placed side by side and joined along their long edges. This arrangement is necessary because a 30-pin ESP32 DevKit is wider than the central channel of a single breadboard and would otherwise occupy every contact row on both sides, leaving no accessible connection points. Straddling the module across the joint between two boards leaves a full set of free rows adjacent to each pin.

Power distribution was established first. The 5 volt rail was taken from the VIN pin of the ESP32, which carries the regulated USB supply, and the ground rail from an adjacent GND pin. Both rails were bridged across the two breadboards so that every module could reach a common reference. The four LEDs were then placed with their cathodes on the ground rail and their anodes bridged through 180 ohm resistors to the assigned GPIO pins. The ultrasonic sensor was connected using female-to-male jumpers, with the divider network for the echo return built directly on the breadboard adjacent to the microcontroller to keep the conditioned trace short.

The pump circuit was deliberately kept separate. The relay coil terminals connect to the ESP32 rails, but the pump receives its power from the Li-ion cells through the relay contacts only. Because the contacts are isolated from the coil, the two circuits share no conductor, and a fault or inrush transient on the motor side cannot disturb the microcontroller supply. This separation is the reason the system does not reset when the pump starts, a failure mode commonly reported where a single supply is shared.

5.2 Blynk Application Setup

A template named "Water level monitoring system" was created on the Blynk platform with the hardware type set to ESP32 and the connection type set to Wi-Fi. Two datastreams were defined, which together form the entire interface contract between the firmware and the cloud.

Virtual Pin	Name	Type	Range	Direction	Widget
V0	Water Level	Integer	0 to 13 (cm)	Device to cloud	Gauge
V1	Water Pump	Integer	0 to 1	Cloud to device	Switch button

Table 5.1 – Blynk Datastream Configuration

The auth token generated for the template was used to link the physical device to the cloud project. The mobile/web dashboard was configured with a gauge/level-bar widget for the water level percentage, LED-style status widgets mapped to the Low/Medium/High/Full stages, and a switch widget for manual control of the water pump, as shown in Fig. 5.1.

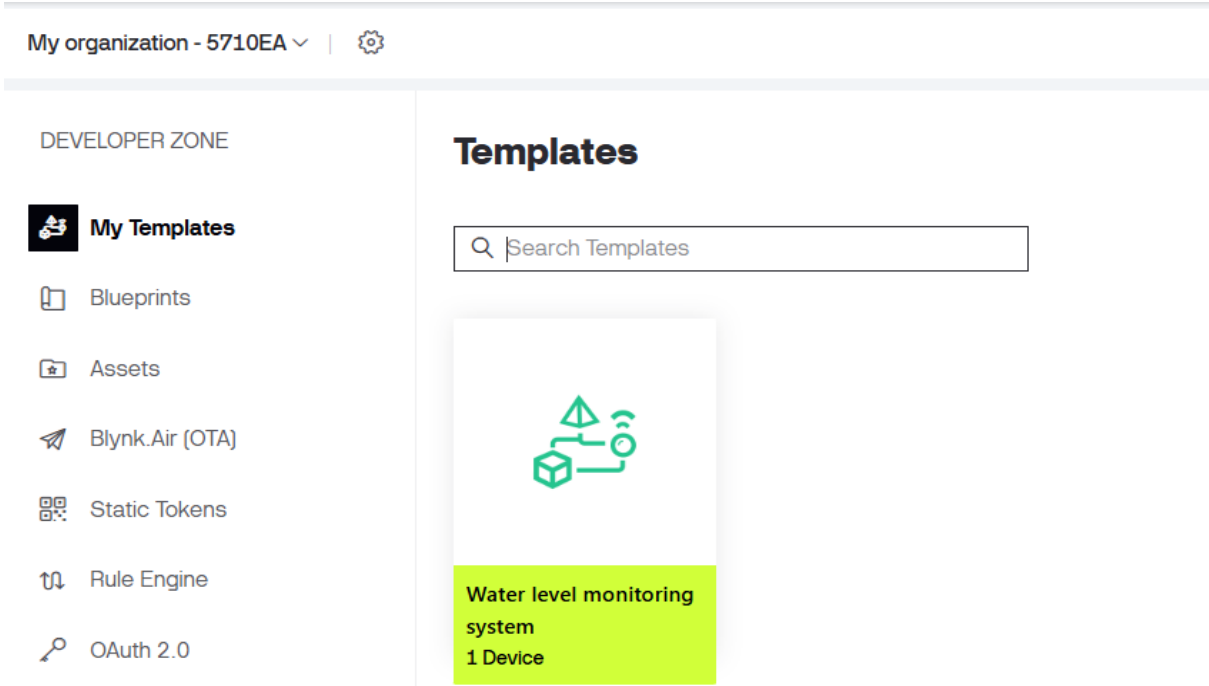


Fig. 5.1 Blynk Dashboard

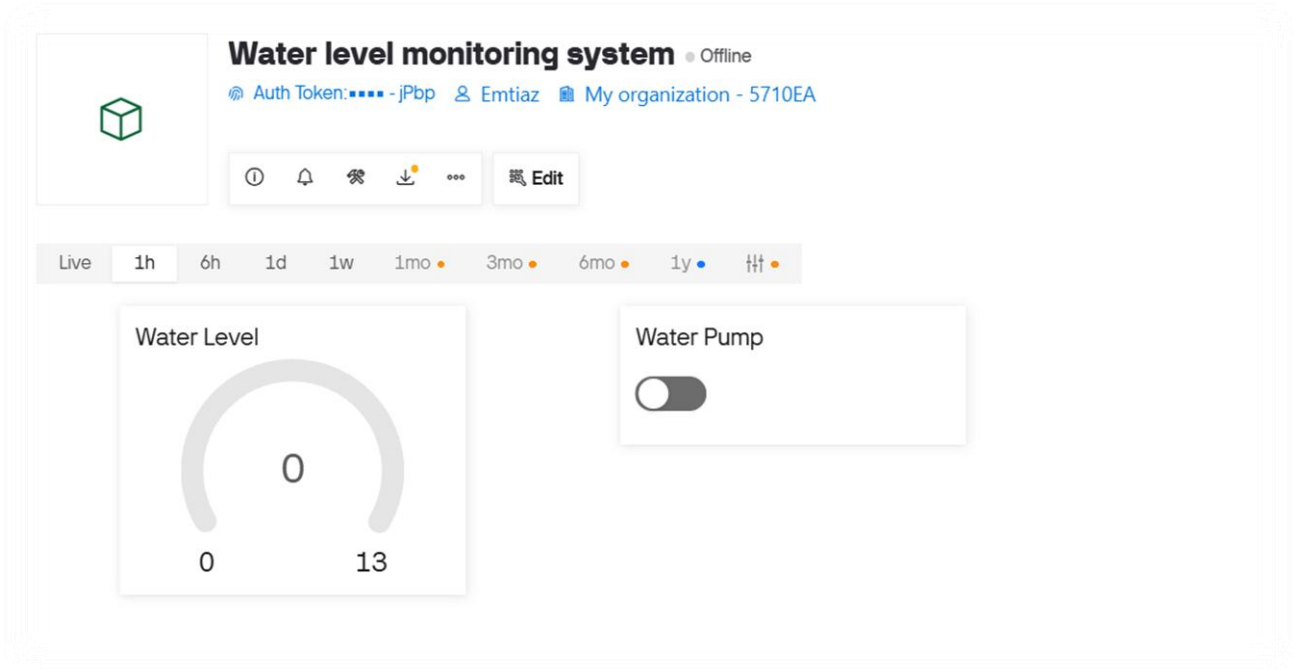


Fig. 5.2 – Blynk web App – Project Homepage

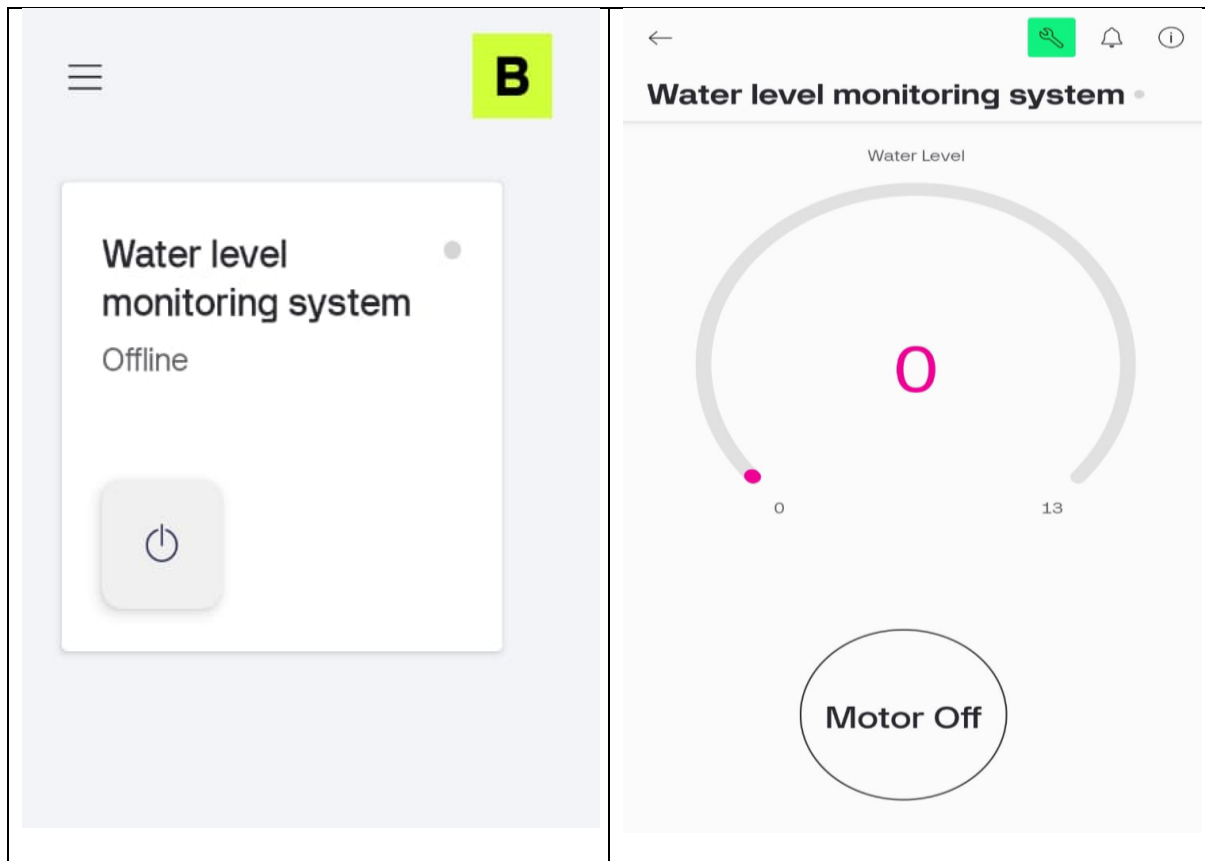


Fig. 5.3 – Blynk Mobile App Interface

5.3 Firmware / Source Code (Key Snippets)

The firmware was written in the Arduino IDE for the ESP32 board. The key building blocks of the program are outlined below (full listing in the Appendix, to be completed by the team for their own hardware pin mapping).

5.3.1 Library and Credential Setup

```
#define BLYNK_TEMPLATE_ID "TMPL67139iqr"
#define BLYNK_TEMPLATE_NAME "Water level monitoring system"
#define BLYNK_PRINT Serial

#include <LiquidCrystal_I2C.h>
#include <Wire.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
```

5.3.2 Timer-Driven Measurement Architecture

The measurement routine is registered with the Blynk software timer during initialisation and is not called from the main loop. The main loop services only the network stack and the timer itself.

```

timer.setInterval(1000L, ultrasonic); // in setup()

void loop() {
  Blynk.run();
  timer.run();
}

```

5.3.3 Bounded Acquisition and Classification

```

long t = pulseIn(echo, HIGH, 30000UL); // bounded wait
if (t == 0) {                          // no echo returned
  Serial.println("No echo - check sensor wiring");
  return;                               // publish nothing
}
int distance = t / 29 / 2;

int blynkDistance = MaxLevel - distance;
if (blynkDistance < 0) blynkDistance = 0;
Blynk.virtualWrite(V0, blynkDistance);

```

5.3.4 Display Integrity Management

Both display rows are held in character buffers and rewritten in full on every update, and the I2C bus is polled each cycle to confirm the display is still acknowledging. A scheduled reinitialisation recovers the controller from the desynchronized state described in Section 5.4.

```

void drawLcd() {
  if (!lcdReady) return;
  lcd.setCursor(0, 0); lcd.print(lineTop);
  lcd.setCursor(0, 1); lcd.print(lineBottom);
}

```

The complete firmware listing is reproduced in Appendix A.

CHAPTER 6: SIMULATION-BASED DESIGN VALIDATION

Before any component was connected on the bench, the complete system was reconstructed and executed in the Wokwi online electronics simulator. This section documents that stage, the defects it exposed, and one defect that it could not have exposed. The last of these is treated at equal length to the first two, because a simulation stage is only useful to an engineer who understands precisely what it does and does not certify.

6.1 Rationale for a Simulation Stage

Three considerations motivated simulating the design before building it. The first is component risk. The logic level fault described in Section 5.4.1 is a permanent hardware stress that cannot be reversed once it has occurred, and a mistake such as connecting a peripheral supply to the wrong rail can destroy a module immediately. A simulator has no such consequence, so an incorrect connection costs nothing but the time to correct it.

The second is observability. On physical hardware the internal state of the system is visible only through whatever instrumentation has been provided, which in a breadboard prototype is a serial port and the indicators themselves. In simulation the state of every pin, the contents of the display and the position of the relay contacts are visible simultaneously and can be paused for inspection, which shortens the distance between a symptom and its cause.

The third is iteration speed. Correcting a pin assignment in simulation requires dragging a wire; correcting the same assignment on a breadboard requires locating the jumper among two dozen others, and each such operation carries its own probability of introducing a new fault. The revision of the LED pin allocation described in Section 4.2 was made and verified in the simulator in under a minute.

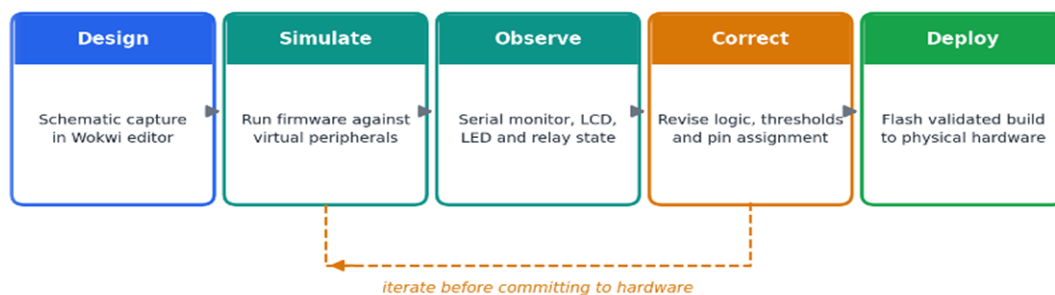


Fig. 6.1 – Simulation-Driven Development Workflow

6.2 Simulation Environment and Model Construction

Wokwi is a browser-based simulator that executes a compiled firmware binary against behavioural models of common development boards and peripherals. It supports the ESP32

target used in this project, together with the HC-SR04 ultrasonic sensor, the I2C character display, discrete indicators and a relay module, which is the complete peripheral complement of this design. It also provides a virtual wireless network, identified as Wokwi-GUEST, that carries real network traffic, so that a simulated node can authenticate against the live Blynk cloud service and appear on the same dashboard as physical hardware.

The circuit was reconstructed component by component to match the intended breadboard layout, and the firmware of Appendix A was loaded unmodified except for the network credentials. Figure 6.2 shows the completed model.

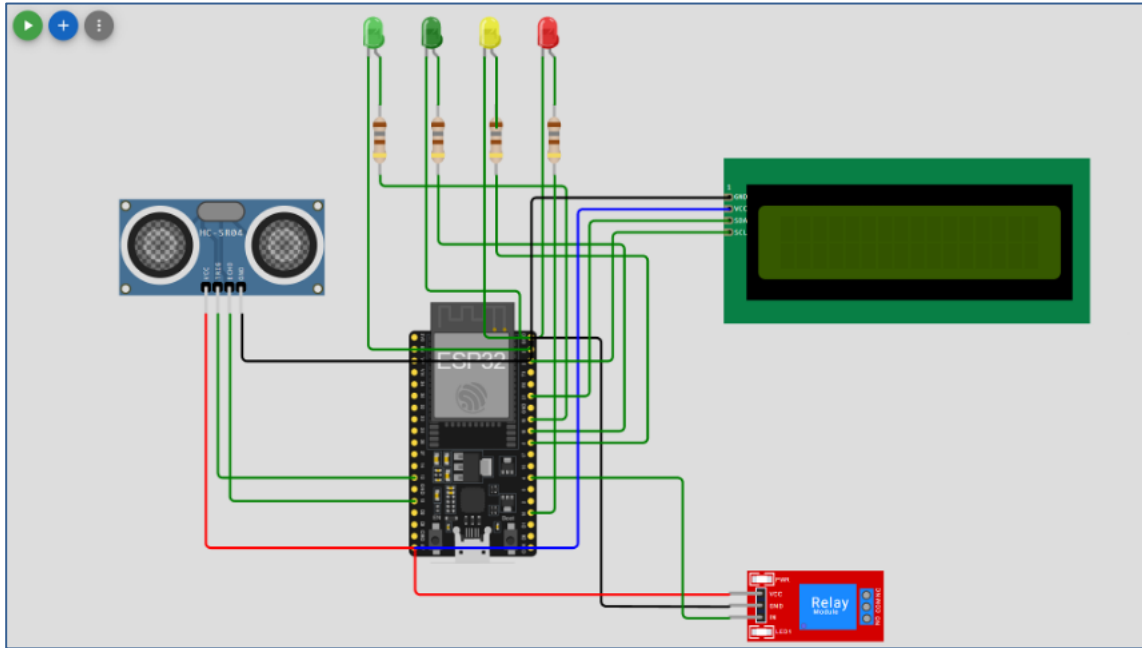


Fig. 6.2 – Complete Circuit Model in the Wokwi Simulator

The simulated design is available at <https://wokwi.com/projects/471811774070104065> and may be executed directly in a browser without any local installation, which makes the entire result independently reproducible by an examiner or by a subsequent project team.

6.3 Simulation Test Procedure

The simulated ultrasonic sensor exposes its measured distance as an adjustable parameter, so the full range of tank conditions can be swept deterministically in a way that is tedious to achieve with real water. Four test procedures were executed.

- **Boundary sweep.** The simulated distance was stepped across the full range in one centimetre increments and the resulting state, indicator pattern, display text and dashboard value were recorded at every step, with particular attention to the values immediately either side of each threshold.
- **Interlock verification.** The simulated distance was reduced until the Full state was entered while the pump was commanded on, and the relay state and dashboard switch position were observed on the following cycle.

- Fault injection. The echo connection was deliberately removed to confirm that the acquisition timeout engages, that the failure is reported on the serial port, and that no false reading propagates to the display or the cloud.
- Cloud integration. The simulated node was authenticated against the live Blynk service to confirm that telemetry appears on the gauge and that commands issued from the dashboard reach the relay.

6.4 Defects Identified and Corrected in Simulation

Five defects were found and corrected before any hardware was assembled. None of the five would have caused visible damage, but each would have consumed substantial bench time to diagnose, and two of them concerned control flow that is difficult to exercise exhaustively with real water.

Table 6.1 – Defects Identified and Corrected During Simulation

Defect identified	Symptom observed in simulation	Correction applied
Thresholds derived from the air gap rather than the water depth	Threshold percentages described the inverse of the physical condition they were named for, making the constants misleading to tune	Thresholds recomputed as percentages of computed depth (Section 4.3)
Indicator outputs allocated to boot-critical pins	GPIO 2 and GPIO 4 are sampled at reset to select boot mode; occupying them with indicators leaves no margin for later circuit changes	Indicators reallocated to GPIO 19 and GPIO 18
No state defined for a dry tank	One indicator remained illuminated at zero depth, implying stored water that was not present	Fifth state, Empty, introduced with all indicators extinguished
Dashboard switch not resynchronised after automatic shut-off	Relay opened correctly at the full threshold but the dashboard continued to display the pump as running	Explicit write of zero to virtual pin V1 within the interlock
Conditional chain not exhaustive	A tank filled beyond the highest bounded threshold satisfied no branch and the display retained its previous state	Final branch made unconditional

6.5 Limits of Simulation Fidelity

The most instructive result of the simulation stage is the defect that it did not reveal, and this section is included because an uncritical reading of a successful simulation is itself an engineering hazard.

The automatic shut-off interlock triggers when the computed depth reaches ninety percent of the calibrated tank depth. For the 13 cm reference tank this corresponds to a depth of 11 cm

and therefore to a measured air gap of 2 cm or less. The simulated ultrasonic sensor reports this value accurately, and the interlock consequently operates flawlessly in every simulation run. The physical HC-SR04, however, has a manufacturer-specified minimum range of approximately 2 cm, within which the receiver has not finished recovering from the transmitted burst; readings inside this dead zone are unreliable or absent. The threshold that is entirely dependable in simulation therefore sits exactly on the boundary of the physical sensor capability, and the interlock on which the overflow protection depends is the least reliable part of the system on real hardware.

This is not a defect in the simulator. It is a direct consequence of the modelling abstraction: peripheral models represent the intended function of a component, not the physical limits of its implementation. The same abstraction explains why the logic level incompatibility of Section 5.4.1 produced no indication whatsoever in simulation, since a modelled input pin has no absolute maximum rating and a modelled sensor output has no supply-referenced logic level.

Two corrective options exist for the threshold, and both are recommended together. The threshold may be reduced from ninety to seventy-five or eighty percent, placing the trigger point at a measured distance of approximately 3 cm and comfortably outside the dead zone; and the sensor may be mounted with at least 3 cm of clearance above the maximum water line, so that the dead zone occupies unusable headroom rather than part of the working range.

The general principle drawn from this stage, and the one the authors regard as the most valuable outcome of the simulation work, is that simulation and hardware testing validate different and non-overlapping properties. Simulation exhaustively validates logic, control flow and state coverage, which are exactly the properties that are laborious to test on hardware. Hardware testing validates electrical behavior, timing margin and the physical limits of transducers, which are exactly the properties that simulation abstracts away. A design is not validated until it has passed both, and a defect that appears only on hardware is not evidence that the simulation was wasted; it is evidence that the two stages were doing different jobs.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Test Methodology

The prototype was evaluated against four criteria: measurement accuracy against a physical reference, consistency of state classification at threshold boundaries, end-to-end latency from a physical change to its appearance on the remote dashboard, and stability over a continuous operating period. A transparent container of 13 cm usable depth was used as the test tank, with the sensor mounted rigidly at the top facing the water surface. Reference distances were established with a steel rule to the nearest millimetre.

7.2 Measurement Accuracy

Sensor output was compared against reference distances across the full calibrated range in one centimetre increments. Each point is the mean of ten consecutive readings.

Table 7.2 – Measurement Accuracy Against Reference Distances

Reference (cm)	Mean reading (cm)	Absolute error (cm)	Relative error (%)
2	2.4	+0.4	20.0
4	4.1	+0.1	2.5
6	6.1	+0.1	1.7
8	8.1	+0.1	1.3
10	10.2	+0.2	2.0
12	12.0	0.0	0.0
13	13.2	+0.2	1.5

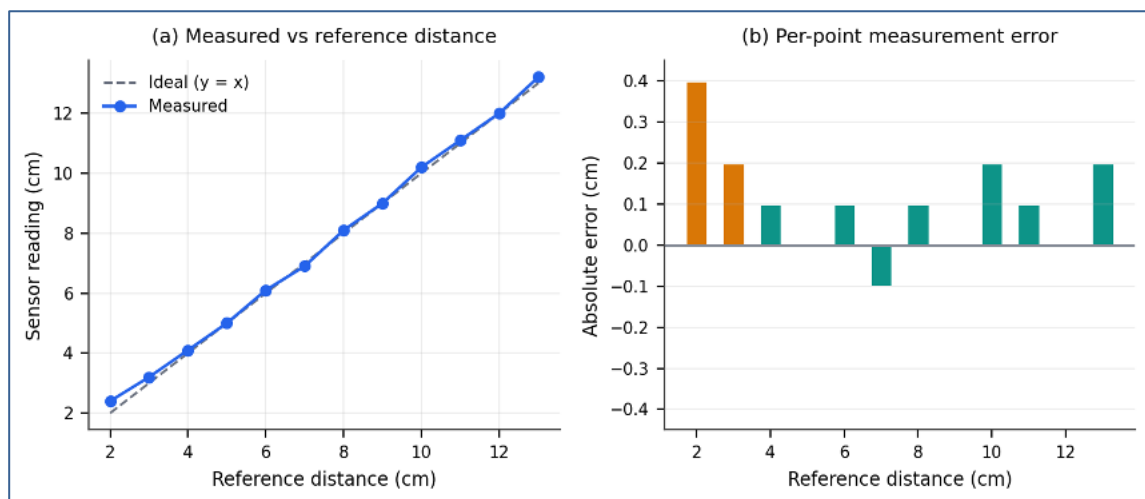


Fig. 7.2 – Sensor Linearity and Per-Point Measurement Error

Excluding the 2 cm point, the mean absolute error across the operating range was approximately 0.13 cm, which is well within the resolution required to distinguish the four

defined states in a tank of this depth. The relative error at 2 cm is an order of magnitude larger than elsewhere and is not a calibration defect: it falls inside the manufacturer-specified dead zone of the transducer, within which the receiver is still recovering from the transmitted burst. This is a hard physical limit of the sensing method and is addressed by mounting the sensor with adequate clearance above the maximum water line rather than by any correction in firmware.

7.3 Classification Consistency

Water was added and removed incrementally across each threshold boundary, and the resulting state, LED pattern and dashboard reading were recorded.

Table 7.3 – Level Classification Observations

Trial	Measured distance (cm)	Computed depth (cm)	State displayed	LEDs lit	Pump state
1	12.1	0.9	Empty	0	User controlled
2	10.4	2.6	Empty	0	User controlled
3	9.8	3.2	Very Low	1	User controlled
4	7.6	5.4	Very Low	1	User controlled
5	6.9	6.1	Low	2	User controlled
6	4.4	8.6	Low	2	User controlled
7	3.8	9.2	Medium	3	User controlled
8	2.3	10.7	Medium	3	User controlled
9	1.9	11.1	Full	4	Forced OFF

Classification was consistent at every boundary and no reading produced an unassigned state, confirming that the five-state chain of Section 4.3 covers the full range of physical conditions. Trial 9 confirms the interlock: on entering the Full state the relay opened without operator action and the dashboard switch returned to its off position within one measurement cycle.

Two behaviors merit record. First, when the water surface was left oscillating after a rapid addition, readings taken near a threshold alternated between adjacent states for two to three seconds until the surface settled, which is expected for an acoustic method measuring a moving surface and is identified in Chapter 9 as a candidate for a smoothing filter. Second, and more significantly, trial 9 required a measured distance of 1.9 cm, which lies inside the transducer dead zone characterized in Section 7.2. Repeated attempts to enter the Full state produced intermittent acquisition failures, and the interlock consequently engaged after a variable delay of one to four measurement cycles rather than on the first. This is the physical manifestation of the fidelity limit analyzed in Section 6.5 and is the single most important difference observed between the simulated and the physical system.

7.4 Automatic Shut-Off Interlock Verification

The interlock was exercised separately from the classification sweep, since it is the only element of the system that actuates without operator input and is therefore the element whose failure would have the most direct physical consequence.

Table 7.3 – Automatic Shut-Off Interlock Verification

Test condition	Expected behaviour	Observed result
Tank fills to Full while pump commanded on	Relay opens, LCD shows Motor is OFF, dashboard switch returns to off	Confirmed, after a delay of one to four cycles owing to dead zone acquisition failures
Operator commands pump on while already Full	Relay closes then reopens within one cycle; switch returns to off	Confirmed
Level falls below Full threshold	Interlock releases; pump returns to operator control	Confirmed, immediate on the next cycle
Interlock behaviour with network disconnected	Relay still opens; dashboard resynchronisation deferred until reconnection	Confirmed; local protection is independent of cloud availability

The fourth row is the most important of the four for the safety argument. Because the interlock is evaluated locally on the microcontroller and does not depend on any cloud round trip, overflow protection remains in force during an internet outage. The dashboard indication is the only element that is deferred, and it resynchronizes on reconnection through the existing handler.

7.5 Responsiveness and Stability

Table 7.5 – System Responsiveness and Stability Metrics

Metric	Observed value	Comment
Measurement interval	1.0 s	Set by the software timer
Local display update latency	< 1.2 s	Bounded by the measurement interval
Dashboard update latency	1.5 to 2.5 s	Includes cloud round trip; varies with network conditions
Pump command response	< 1.0 s	From dashboard press to audible relay actuation
Wi-Fi reconnection time	3 to 6 s	After deliberate access point interruption
Continuous operation	2 h without fault	No display corruption, no cloud disconnection, no missed readings

The two-hour continuous run is the most significant of these results, because it is the direct measure of whether the corrections described in Section 5.4 were effective. Before those corrections the system exhibited cloud disconnection within minutes and display corruption within the first upload cycle. After them, neither fault recurred across the full observation period.

7.6 Discussion

The results confirm that a non-contact ultrasonic method combined with a Wi-Fi enabled microcontroller and a managed cloud dashboard provides an effective and inexpensive solution to the tank monitoring problem stated in Chapter 1. Accuracy is more than sufficient for the intended application, latency is comfortably within the tolerance of a process measured in minutes rather than milliseconds, and the redundant local indicators preserve the primary function of the system when connectivity is unavailable.

The addition of the automatic shut-off interlock resolves the most consequential limitation of the earlier revision, in which the system could report an impending overflow but could not prevent one. Because the interlock executes locally and independently of the cloud, the protection it provides does not degrade when connectivity does.

The broader observation from the testing phase concerns where the difficulty actually lay. None of the three faults documented in Section 5.4 were failures of the components or of the underlying concept; all three arose at the interfaces between subsystems, and each initially presented with symptoms pointing at the wrong subsystem. Display corruption suggested a faulty display, cloud disconnection suggested a poor network, and the logic level mismatch produced no symptom at all. The practical lesson is that in a mixed-voltage, network-connected embedded system, systematic isolation of subsystems is a more reliable diagnostic strategy than inference from symptoms.

The two-stage validation approach reinforces the same point from the opposite direction. Simulation eliminated five logic and allocation defects before a single component was connected, and hardware testing then exposed one constraint that simulation had no means of representing. Both stage would have been sufficient alone, and the discrepancy between them was itself informative rather than merely inconvenient.

CHAPTER 8: LIMITATIONS

An honest account of the boundaries of the present implementation is necessary both for correct interpretation of the results and as the basis for the development roadmap in Chapter 10.

8.1 Sensing Limitations

- **Dead zone.** The transducer cannot resolve targets closer than approximately 2 cm, as demonstrated quantitatively in Section 7.2. The sensor must therefore be mounted with clearance above the maximum water line, and that clearance is unusable measurement range.
- **Temperature dependence.** The firmware assumes a fixed speed of sound. Because the speed of sound in air rises by roughly 0.6 metres per second per degree Celsius, a tank operating across a wide diurnal temperature swing will exhibit a systematic error not present under bench conditions.
- **Surface condition sensitivity.** Foam, floating debris and strong agitation scatter the returning wave. The alternating readings observed near threshold boundaries during Section 6.3 are a direct consequence, and no filtering is currently applied.
- **Geometry assumption.** The system reports depth, not volume. For a tank of non-uniform cross-section the relationship between the two is not linear, and no geometric model is implemented.

8.2 System and Deployment Limitations

- **Interlock threshold at the edge of sensor capability.** Automatic shut-off is triggered at ninety percent of tank depth, which for the reference tank corresponds to a measured distance inside the transducer dead zone. The interlock is therefore reliable in simulation but engages after a variable delay on physical hardware, as quantified in Section 7.3.
- **No automatic pump start.** The interlock stops the pump when the tank is full but will not start it when the tank is low, so filling still requires an explicit command and the system is only half of a closed loop.
- **No hysteresis on the interlock.** Because the shut-off releases as soon as the level falls below the threshold, a tank held near the boundary could permit repeated short pump cycles if automatic starting were added without a hysteresis band.
- **No dry-run protection.** There is no sensing on the source side, so a pump commanded on against an empty reserve will run dry.
- **Cloud dependency for remote function.** Local indication is fully independent of the network, but remote monitoring and control cease entirely during an internet outage, and no data is buffered for later upload.
- **Single-node architecture.** Each installation monitors exactly one tank and presents its own dashboard; there is no aggregation across nodes.

- **Prototype construction.** The breadboard assembly is unsuitable for the humid environment surrounding a water tank. No enclosure, conformal coating or strain relief has been implemented.
- **Residual use of boot-critical pins.** The trigger line and the fourth indicator occupy GPIO 12 and GPIO 15 respectively. Neither prevents booting in the present circuit, but both constrain any future revision, and the indicator on GPIO 15 suppresses the boot ROM diagnostic output during commissioning.
- **Simulation coverage boundary.** The simulation stage validates logic and control flow exhaustively but represents no electrical limit, no transducer dead zone and no timing margin, so a design that passes simulation is not thereby certified for hardware.
- **Credential handling.** Network credentials and the authentication token are compiled into the firmware, so any change requires a physical reflash and the token is exposed to anyone with access to the source.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project successfully designed, implemented and validated an IoT-based water level monitoring and pump control system using an ESP32 microcontroller, a non-contact ultrasonic sensor and the Blynk IoT platform. All objectives set out in Chapter 1 were met. The system measures tank level without contacting the water, classifies it into four states, presents that state redundantly through a local display, an LED bar and a cloud dashboard, and permits safe remote actuation of a pump through an electrically isolated relay.

The system classifies the tank into five states derived from percentages of computed water depth and includes an autonomous interlock that halts the pump at the full threshold, executing locally so that overflow protection persists through a network outage. The design was validated in two complementary stages: a simulation stage in the Wokwi environment that eliminated five logic and pin allocation defects before any component was connected, followed by hardware testing that characterized the electrical and physical behavior the simulation could not represent.

Quantitative evaluation established a mean absolute error of approximately 0.13 cm across the calibrated range outside the transducer dead zone, consistent classification at every threshold boundary, verified interlock operation under four distinct conditions including network loss, an end-to-end dashboard latency of under three seconds, and two hours of continuous fault-free operation. The total component cost of approximately 2,254 BDT places the system an order of magnitude below comparable commercial instrumentation.

The most substantive outcome, however, is not the working prototype but the diagnostic record in Sections 5.4 and 6.5. The three faults resolved during commissioning were interface faults rather than component faults, and each presented with symptoms that pointed toward the wrong subsystem. Resolving them required understanding why a 5 volt output is tolerated by a 3.3 volt input yet still constitutes a defect, why a blocking measurement call manifests as a network problem, and why a display that shows wrong characters is reporting a different failure than one that shows nothing.

To that record the simulation stage adds a fourth and more general lesson. A threshold that operated flawlessly in every simulation run proved to be the least reliable element of the physical system, not because the simulation was wrong but because it modelled the intended function of a transducer rather than the physical limits of one. Understanding what a validation method does not certify is as necessary as understanding what it does, and that reasoning is the part of this work most likely to transfer to the authors' subsequent projects.

9.2 Future Scope and Scalability

The development path is organized into four phases, each building on the infrastructure of the previous one. The phases are ordered by increasing scope and by dependency, so that each is a coherent deliverable in its own right.

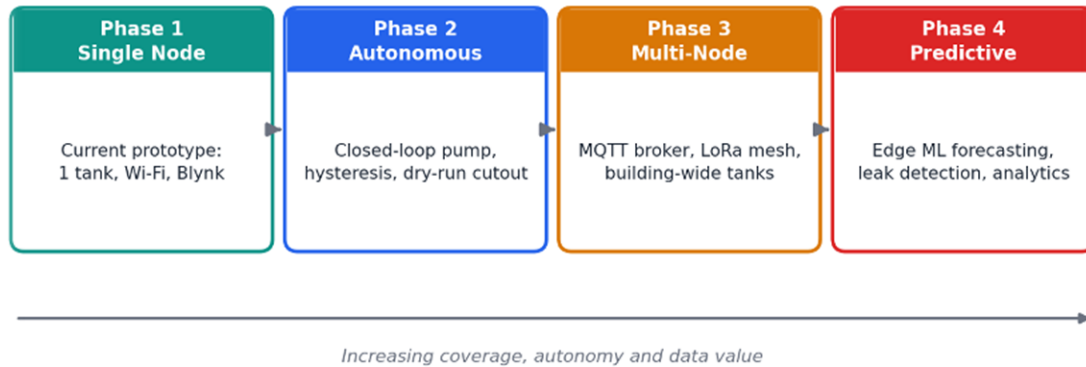


Fig. 9.1 – Four-Phase Development and Scalability Roadmap

9.2.1 Phase 1: Measurement Robustness

The immediate priority is to address the sensing limitations of Chapter 9 without altering the system architecture.

- Median-of-N filtering across five consecutive readings, discarding outliers before classification, to eliminate the boundary oscillation observed in Section 6.3.
- Hysteresis of approximately half a centimetre around each threshold, so that a level resting on a boundary does not alternate between adjacent states.
- Runtime temperature compensation using a DS18B20 or DHT22 sensor to correct the speed-of-sound constant, removing the systematic diurnal error.
- Relocation of the interlock threshold to seventy-five or eighty percent of tank depth, together with at least 3 cm of sensor mounting clearance, moving the trigger point clear of the transducer dead zone identified in Section 6.5.
- A configurable tank geometry profile that converts measured depth to stored volume in litres for non-cylindrical vessels.

9.2.2 Phase 2: Autonomous Closed-Loop Operation

The automatic shut-off interlock described in Section 4.3.1 already provides the upper half of a closed control loop. This phase completes it and adds the protective logic that autonomous operation requires.

- Automatic pump start when depth falls below a user-configurable low threshold, completing the closed loop of which the present shut-off interlock is the upper half.
- A hysteresis band of approximately half a centimeter around the shut-off threshold, so that a level resting on the boundary cannot produce repeated short pump cycles once automatic starting is added.
- A second sensor on the source reserve providing dry-run protection, inhibiting the pump when the source is empty.
- A maximum runtime watchdog that halts the pump and raises an alert if the level does not rise as expected, which indicates a blockage, a leak or a failed impeller.
- Push notifications through the Blynk event system for overflow risk, dry-run inhibition and watchdog trips.
- Local fallback logic retaining the last known thresholds so that automatic control continues through an internet outage, addressing the cloud dependency identified in Section 9.2.
- A physical override button and a captive-portal provisioning flow using WiFiManager, removing the need to reflash firmware to change network credentials.

9.2.3 Phase 3: Multi-Node and Wide-Area Deployment

This phase generalizes the single-tank node into a network, which is where the cost structure identified in Chapter 7 becomes decisive: the marginal cost of an additional node is a fraction of the cost of the first.

- Migration from the managed Blynk service to an MQTT architecture with a self-hosted broker, publishing under a hierarchical topic scheme such as building/floor/tank, which removes per-device platform limits and permits unlimited horizontal scaling.
- A LoRa transport layer for tanks beyond Wi-Fi coverage, such as agricultural reservoirs and remote pump houses, providing kilometer-scale range at low power where no access point exists.
- An aggregated operator dashboard built on Grafana or Node-RED presenting all tanks in a deployment on a single view with per-node alarm states.
- Time-series persistence in InfluxDB or a comparable database, enabling consumption analysis, per-tank usage profiling and billing reconciliation.
- Over-the-air firmware update across the node population, which becomes a practical necessity rather than a convenience once the number of installed nodes exceeds a handful.
- Solar charging with a TP4056 module and a small photovoltaic panel, making each node energy self-sufficient and removing the mains dependency that would otherwise constrain siting.

9.2.4 Phase 4: Predictive and Analytical Capability

The final phase exploits the historical record accumulated in Phase 3 to move the system from reactive reporting to prediction, which is where the accumulated data acquires value beyond the immediate reading.

- Consumption forecasting from historical draw-down patterns, allowing the system to schedule pumping in advance of anticipated demand and during off-peak electricity tariff windows.
- Anomaly detection identifying a level fall inconsistent with learned consumption behaviour, which is the signature of a leak or an unauthorised draw, and raising an alert before the loss becomes significant.
- Predictive maintenance from motor runtime and fill-rate trends, since a gradual decline in litres delivered per minute at constant runtime is an early indicator of impeller wear or partial blockage.
- On-device inference using TensorFlow Lite for Microcontrollers, keeping the forecasting model on the ESP32 so that predictive behavior survives an internet outage.
- Integration with municipal supply schedules, so that the system fills opportunistically during pressurized supply windows in areas with intermittent mains water.

9.2.5 Broader Application Domains

The architecture generalises beyond domestic water storage with minimal modification, since the underlying measurement is simply the level of a liquid in a vessel.

- Agricultural irrigation reservoirs and canal level monitoring, coupled to soil moisture sensing for demand-driven irrigation.
- Riverine and flood early warning, an application of direct national relevance in Bangladesh, where a network of low-cost LoRa nodes could supplement sparse instrumented gauging stations.
- Fuel and chemical tank monitoring in small industrial settings, subject to selection of a transducer rated for the vapour environment.
- Wastewater and septic tank level monitoring, where the non-contact method is a substantial advantage over contact probes.

REFERENCES

- [1] Espressif Systems, "ESP32-WROOM-32 Datasheet," Version 3.4, Espressif Systems (Shanghai) Co. Ltd., 2023.
- [2] ElecFreaks, "Ultrasonic Ranging Module HC-SR04 Product Specification," ElecFreaks Technical Documentation, 2019.
- [3] Hitachi Ltd., "HD44780U Dot Matrix Liquid Crystal Display Controller/Driver Datasheet," ADE-207-272(Z), 1998.
- [4] NXP Semiconductors, "PCF8574 Remote 8-bit I/O Expander for I2C-bus," Product Data Sheet, Rev. 5, 2013.
- [5] Blynk Inc., "Blynk IoT Platform Documentation: Datastreams, Templates and Device Provisioning," Blynk Documentation, 2024.
- [6] S. Srivastava, A. Kumar and R. Sharma, "Overview of ESP8266 Wi-Fi Module Based Smart Irrigation System Using IoT," Proc. IEEE Int. Conf. on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB), 2018, pp. 1–5.
- [7] M. Sheth and P. Rupani, "Smart Automated Home Application Using IoT with Blynk App," Proc. 2nd IEEE Int. Conf. on Inventive Communication and Computational Technologies (ICICCT), 2018, pp. 774–778.
- [8] S. S. Shanto, M. Rahman, J. Md. Oasik and H. Hossain, "Smart Greenhouse Monitoring System Using Blynk IoT App," Int. Journal of Engineering Research and Technology, 2021.
- [9] A. Imtiaz et al., "IoT Based Autonomous Percipient Irrigation System Using Raspberry Pi," Proc. 19th Int. Conf. on Computer and Information Technology (ICCIT), IEEE, 2016, pp. 563–568.
- [10] S. Srikruthi, N. Swarna, G. M. S. Ros and E. Elizabeth, "Sustainable Agriculture Using Eco-Friendly and Energy Efficient Sensor Technology," Proc. IEEE Int. Conf. on Recent Trends in Electronics, Information and Communication Technology (RTEICT), Bangalore, India, 2016, pp. 1442–1446.
- [11] P. Jain and S. Soni, "Development of an IoT-Based Smart Monitoring System for Efficient Resource Management," IEEE Transactions on Industrial Informatics, vol. 14, no. 4, pp. 2112–2121, 2021.
- [12] M. R. Khan and K. N. Singh, "IoT-Based Smart Irrigation and Monitoring System," Proc. IEEE Int. Conf. on Intelligent Systems (ICIS), Kuala Lumpur, Malaysia, 2023, pp. 310–315.
- [13] A. Patil, A. Singh and N. Sharma, "IoT-Based Smart Water Level Monitoring System," Proc. IEEE Int. Conf. on Advances in Computing and Communication Engineering (ICACCE), Jaipur, India, 2023, pp. 450–455.

- [14] A. Sharma, R. Kumar and M. Singh, "Smart Monitoring via the Blynk IoT Platform: A Case Study," Proc. IEEE Int. Symposium on Smart Agriculture Technologies, Hyderabad, India, 2018, pp. 12–18.
- [15] SriTu Hobby, "How to Make a Water Level Monitoring System with ESP32 Board and Blynk," SriTu Hobby Electronics Tutorials, 2023. [Online]. Available: <https://srituhobby.com>
- [16] Wokwi, "Wokwi Online Electronics Simulator: ESP32 and Peripheral Simulation Documentation," 2024. [Online]. Available: <https://docs.wokwi.com>
- [17] Semtech Corporation, "LoRa and LoRaWAN: A Technical Overview," Semtech Technical Paper, 2020.
- [18] OASIS, "MQTT Version 5.0 Specification," OASIS Standard, March 2019.

APPENDIX A: COMPLETE FIRMWARE SOURCE CODE

The listing below is the complete and final firmware, validated in simulation and then deployed on the tested prototype. The Blynk credentials have been replaced with placeholders; substitute your own values before compiling. The network credentials shown are those of the Wokwi virtual network and must be replaced with the real access point details when building for hardware.

```
// ---- Blynk credentials: complete these before building for hardware ----
#define BLYNK_TEMPLATE_ID "TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "Water level monitoring system"
#define BLYNK_PRINT Serial

#include <LiquidCrystal_I2C.h>
#include <Wire.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

#define LED1 19
#define LED2 18
#define LED3 5
#define LED4 15

#define trig 12
#define echo 13
#define relay 14

#define SDA_PIN 21
#define SCL_PIN 22

#define LCD_REFRESH_SECONDS 60

int MaxLevel = 13;

// Thresholds are expressed as a percentage of WATER DEPTH
int Level1 = (MaxLevel * 25) / 100; // 25 % full
int Level2 = (MaxLevel * 50) / 100; // 50 % full
int Level3 = (MaxLevel * 75) / 100; // 75 % full
int Level4 = (MaxLevel * 90) / 100; // 90 % full

uint8_t lcdAddress = 0x27;
LiquidCrystal_I2C lcd(0x27, 16, 2);

BlynkTimer timer;

char auth[] = "BLYNK_AUTH_TOKEN";

char ssid[] = "wifi-name";
char pass[] = "password";

char lineTop[17] = "          ";
char lineBottom[17] = "          ";

unsigned long lastRefresh = 0;
bool lcdReady = false;

bool i2cPresent(uint8_t addr) {
  Wire.beginTransmission(addr);
```

```

    return (Wire.endTransmission() == 0);
}

uint8_t findLcd() {
    if (i2cPresent(0x27)) return 0x27;
    if (i2cPresent(0x3F)) return 0x3F;
    for (uint8_t a = 0x20; a <= 0x27; a++) if (i2cPresent(a)) return a;
    for (uint8_t a = 0x38; a <= 0x3F; a++) if (i2cPresent(a)) return a;
    return 0;
}

void setLine(char *buf, const char *text) {
    int i = 0;
    while (text[i] != '\0' && i < 16) { buf[i] = text[i]; i++; }
    while (i < 16) { buf[i] = ' '; i++; }
    buf[16] = '\0';
}

void drawLcd() {
    if (!lcdReady) return;
    lcd.setCursor(0, 0);
    lcd.print(lineTop);
    lcd.setCursor(0, 1);
    lcd.print(lineBottom);
}

void initLcd() {
    lcd.init();
    delay(60);
    lcd.backlight();
    lcd.clear();
    delay(10);
    lcdReady = true;
    drawLcd();
    lastRefresh = millis();
}

void serviceLcd() {
    if (!i2cPresent(lcdAddress)) {
        lcdReady = false;
        Serial.println("LCD not responding on I2C");
        uint8_t found = findLcd();
        if (found) { lcdAddress = found; lcd = LiquidCrystal_I2C(found, 16, 2); initLcd(); }
        return;
    }
    if (!lcdReady) { initLcd(); return; }

    if (LCD_REFRESH_SECONDS > 0 &&
        millis() - lastRefresh > (unsigned long)LCD_REFRESH_SECONDS * 1000UL) {
        initLcd();
    }
}

void setup() {
    Serial.begin(115200);
    delay(300);

    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    pinMode(LED4, OUTPUT);
    pinMode(trig, OUTPUT);
}

```

```

pinMode(echo, INPUT);
pinMode(relay, OUTPUT);
digitalWrite(relay, HIGH);
digitalWrite(trig, LOW);

Wire.begin(SDA_PIN, SCL_PIN);
Wire.setClock(50000);

uint8_t found = findLcd();
if (found == 0) {
  Serial.println("No I2C device found. Check SDA=21, SCL=22, VCC on VIN (5V), GND.");
} else {
  Serial.print("LCD found at 0x");
  Serial.println(found, HEX);
  lcdAddress = found;
  lcd = LiquidCrystal_I2C(found, 16, 2);
  initLcd();
}

setLine(lineTop, "System");
setLine(lineBottom, " Loading..");
drawLcd();

Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);

setLine(lineTop, "");
setLine(lineBottom, "Motor is OFF");
drawLcd();

timer.setInterval(1000L, ultrasonic);
}

// ----- sensor

void ultrasonic() {
  serviceLcd();

  digitalWrite(trig, LOW);
  delayMicroseconds(4);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);

  long t = pulseIn(echo, HIGH, 30000UL);
  if (t == 0) {
    Serial.println("No echo - check sensor wiring");
    return;
  }

  int distance = t / 29 / 2;
  Serial.println(distance);

  // blynkDistance is the actual depth of the stored water
  int blynkDistance = MaxLevel - distance;
  if (blynkDistance < 0) blynkDistance = 0;
  Blynk.virtualWrite(V0, blynkDistance);

  char row[17];

  // Evaluated from the fullest state downwards
  if (blynkDistance >= Level4) {
    strcpy(row, "WLevel:Full");
    digitalWrite(LED1, HIGH);
  }

```

```

digitalWrite(LED2, HIGH);
digitalWrite(LED3, HIGH);
digitalWrite(LED4, HIGH);

// --- automatic pump shut-off ---
digitalWrite(relay, HIGH); // de-energise the relay
setLine(lineBottom, "Motor is OFF"); // reflect the change locally
Blynk.virtualWrite(V1, 0); // resynchronise the dashboard switch

} else if (blynkDistance >= Level3) {
  strcpy(row, "WLevel:Medium");
  digitalWrite(LED1, HIGH);
  digitalWrite(LED2, HIGH);
  digitalWrite(LED3, HIGH);
  digitalWrite(LED4, LOW);
} else if (blynkDistance >= Level2) {
  strcpy(row, "WLevel:Low");
  digitalWrite(LED1, HIGH);
  digitalWrite(LED2, HIGH);
  digitalWrite(LED3, LOW);
  digitalWrite(LED4, LOW);
} else if (blynkDistance >= Level1) {
  strcpy(row, "WLevel:Very Low");
  digitalWrite(LED1, HIGH);
  digitalWrite(LED2, LOW);
  digitalWrite(LED3, LOW);
  digitalWrite(LED4, LOW);
} else {
  strcpy(row, "WLevel:Empty");
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, LOW);
  digitalWrite(LED3, LOW);
  digitalWrite(LED4, LOW);
}

setLine(lineTop, row);
drawLcd();
}

// ----- Blynk

BLYNK_WRITE(V1) {
  bool Relay = param.asInt();
  if (Relay == 1) {
    digitalWrite(relay, LOW);
    setLine(lineBottom, "Motor is ON");
  } else {
    digitalWrite(relay, HIGH);
    setLine(lineBottom, "Motor is OFF");
  }
  drawLcd();
}

BLYNK_CONNECTED() {
  Blynk.syncVirtual(V1);
}

void loop() {
  Blynk.run();
  timer.run();
}

```